Carry Graphs

Finite labelled digraphs behind integer parts of powers

Ralf Stephan
in collaboration with Claude Code

Draft, October 11, 2026

Contents

1 Why carry graphs **1**
- 1.1 One recurrence . . . 1
- 1.2 The object . . . 2
- 1.3 Covers, and the shape of every theorem . . . 2
- 1.4 What is in the book . . . 3
- 1.5 Lean . . . 3

2 Carry graphs, walks and words **5**
- 2.1 Carry graphs . . . 5
- 2.2 Walks, infinite paths, words . . . 6
- 2.3 Counting paths and words . . . 7
- 2.4 Kőnig's lemma . . . 7

3 Reachability, components, ranks **9**
- 3.1 Reachability . . . 9
- 3.2 The two canonical ranks . . . 10
- 3.3 Ranks and branching . . . 10
- 3.4 Peel ranks . . . 11
- 3.5 Geodesic Kőnig . . . 12

4 Peeling and the core **15**

5 Growth and entropy **17**

6 Labels, languages and sofic shifts **19**

7 Splittings, homomorphisms and products **21**

8 Covers of dynamical systems **23**

9 Affine carry graphs **25**

A Lean dictionary **27**

B What Mathlib and ForMathlib supply **29**

Chapter 1

Why carry graphs

1.1 One recurrence

Let $p > q \geq 1$ be coprime integers, $\xi > 0$ a real number and ν a real shift. Write

$$\xi\Big(\frac{p}{q}\Big)^n + \nu \;=\; x_n + y_n, \qquad x_n \in \mathbb{Z}, \quad y_n \in [0,1),$$

so that $x_n = \lfloor \xi(p/q)^n + \nu \rfloor$ is the integer part and y_n the fractional part. Multiplying the identity at n by p/q and comparing with the identity at $n+1$ gives

$$q\,x_{n+1} \;=\; p\,x_n + s_n, \qquad s_n \;=\; p\,y_n - q\,y_{n+1} - (p-q)\,\nu \in \mathbb{Z}. \tag{1.1}$$

The integer s_n is the *carry* of step n. It takes only finitely many values, because $y_n, y_{n+1} \in [0,1)$, and it is the only information that passes from the fractional part to the integer part. For $q = 1$ and $\nu = 0$ the recurrence is $x_{n+1} = p\,x_n + s_n$ with $s_n \in \{0, \ldots, p-1\}$, and s_n is the n-th base-p digit of ξ after the point.

Many old questions about the integers $\lfloor \xi(p/q)^n \rfloor$ and the fractional parts $\{\xi(p/q)^n\}$ concern the *carry word* $s_0 s_1 s_2 \cdots$:

- whether x_n is composite for infinitely many n [**?**, **?**, **?**];
- whether every x_n can avoid a finite set $\mathcal{P}$ of prime divisors;
- whether all y_n can stay in a short interval (Mahler's Z-number problem [**?**, **?**]).

In each case the question can be asked about finite data: a residue of x_n modulo some M, the cell of y_n in a partition of $[0,1)$ into D intervals, or both. The recurrence (1.1) moves this finite data along a finite labelled digraph whose arc labels are the carries. This book is about that digraph.

1.2 The object

Definition 1.1. A *carry graph* over a label set A is a set V of vertices together with a map $\mathrm{succ}\colon V \times A \to \{\text{finite subsets of } V\}$. There is an arc $v \xrightarrow{a} w$ when $w \in \mathrm{succ}(v, a)$.

The vertex set may be infinite. Only the out-neighbourhoods have to be finite, and that is the condition the rest of the book uses. Forgetting the labels gives an ordinary digraph in the sense of [**?**, §1.2]. Keeping them gives a labelled graph in the sense of [**?**, §3.1]. A finite carry graph is therefore an object of symbolic dynamics, and most of the tools of [**?**] chapters 1–4 apply to it.

Three families of carry graphs recur.

Residue graphs.
Vertices are residues $r \in \mathbb{Z}/M$. Under the label s, the arc $r \xrightarrow{s} (pr + s)q^{-1}$ is present when $\gcd(q, M) = 1$. Here $x_n \bmod M$ walks along this graph and the label is the carry. Deleting the residues divisible by a prime of $\mathcal{P}$ leaves the residues an avoiding orbit may visit.

Cell graphs.
Vertices are the cells $[c/D, (c+1)/D)$, $0 \le c < D$. There is an arc $c \xrightarrow{s} c'$ when some y in cell c has $(p\,y - (p-q)\nu - s)/q$ in cell c'. Then y_n walks along this graph and the label is again the carry.

Products.
The pair $(x_n \bmod M,\ \text{cell of } y_n)$ walks along the *label product* of a residue graph and a cell graph. Its arcs are the pairs of arcs carrying the same label (Chapter 7).

1.3 Covers, and the shape of every theorem

The orbit itself lives in an infinite space: the pairs (x_n, y_n), or the reals ξ. A carry graph X is tied to the orbit by a *cover*. This is an assignment $\rho\colon V \to \mathcal{P}(S)$ of a region of state space to every vertex, such that every step of the dynamics from $\rho(v)$ under carry a lands in $\rho(w)$ for some arc $v \xrightarrow{a} w$. A cover makes every orbit a path of X, and the carry word of the orbit the label of that path. Conversely, X *realises* the dynamics when every finite labelled path is the carry word of some orbit segment.

With these two notions, every result about the problems above has the same shape:

$$\text{cover of the dynamics by } X \ + \ \text{a property of the finite graph } X$$
$$\Longrightarrow \ \text{a property of every orbit.}$$

The graph properties that occur are few:

- every infinite path of X is eventually periodic (*zero entropy*, Chapter 5); combined with the aperiodicity of carry words [**?**, Lemma 2], an avoiding orbit then cannot exist;
- every infinite path leaves a vertex set after a bounded number of steps (*escape*, Chapter 4);
- the number of labelled paths of length n is at most $C\lambda^n$, which bounds an entropy or a Hausdorff dimension (Chapter 8);
- and, in the other direction, the existence of two closed walks of equal length through one vertex (a *horseshoe*). When X realises the dynamics, this produces orbits with positive entropy, and it shows that no proof of the first kind exists for that graph.

A computer search finds the graph and a finite witness for the property, such as a rank function, a peeling sequence, a sub-eigenvector or a pair of closed walks. A proof assistant checks the witness, and the transfer theorems of this book turn the checked property into a statement about orbits. The book develops the graph theory and the transfer theorems once, for all carry graphs. The witnesses and searches belong to the individual problems and appear here only as examples.

1.4 What is in the book

Chapters 2–4 cover walks, Kőnig's lemma, reachability, strong components and their acyclic orderings [**?**, §1.4–1.5]. They also introduce ranks and the peeling of a graph down to its core, the part that carries every infinite path. Chapter 5 counts paths. It proves the dichotomy between polynomial and exponential growth, the equivalence of six descriptions of zero entropy, and the eigenvector bounds that stand in for the Perron–Frobenius theorem [**?**, §4.1–4.4, §5.3]. Chapter 6 studies labels: label languages, the subset construction, follower sets and sofic shifts [**?**, ch. 3]. Chapter 7 covers state splitting, higher-block and line graphs, homomorphisms, and label products [**?**, §2.3–2.4], [**?**, §4.5]. Chapter 8 defines covers and proves the transfer theorems. Chapter 9 specialises to the affine carry graphs of the recurrence (1.1). It proves that the residue–cell graph is a label product and works through the problems listed above as examples.

1.5 Lean

Every definition and theorem in this book is formalised in Lean 4 on top of Mathlib [**?**], in the directory that also holds this book. The two use the same names: a carry graph is the structure `CarryGraph V A` with one

field `succ : V → A → Finset V`. Each chapter ends with a table that matches its numbered results to Lean declarations, and Appendix A collects these tables. Appendix B lists what Mathlib already provided and what had to be built.

Notes

The recurrence (1.1) and the carry word are classical. The aperiodicity of the carry word is [**?**, Lemma 2]. Residue dynamics for $q = 1$ is the method of [**?**] and [**?**]. Cell dynamics and the intervals that confine $\{\xi(p/q)^n\}$ go back to [**?**]. Finite graph witnesses for composite terms of $\lfloor \xi(a/b)^n \rfloor$ appear in [**?**].

Chapter 2

Carry graphs, walks and words

This chapter fixes the language: carry graphs and their constructions, walks and infinite paths, the words a carry graph reads, and how to count them. It ends with Kőnig's lemma, the one place where finite branching is used in its full strength. Everything here is standard [**?**, §1.3, §2.2, §3.1], [**?**, §1.2–1.4]. The only departure from the textbooks is that the vertex set may be infinite.

2.1 Carry graphs

Definition 2.1. Let V and A be sets. A *carry graph* X on V over the alphabet A is a map $\mathrm{succ}_X \colon V \times A \to \{\text{finite subsets of } V\}$. We write $v \xrightarrow{a} w$, or $X \colon v \xrightarrow{a} w$, when $w \in \mathrm{succ}_X(v, a)$, and call it an *arc* with *label* a. The *underlying digraph* has an arc $v \to w$ when $v \xrightarrow{a} w$ for some a.

When A is finite, every vertex has finitely many out-neighbours in the underlying digraph, namely $\mathrm{out}(v) = \bigcup_a \mathrm{succ}(v, a)$. In that case X is *finitely branching*, which is all that Kőnig's lemma needs. No finiteness is assumed for V. The follower automaton of Chapter 6 is a carry graph whose vertex set is infinite in general.

Parallel arcs with the same label are not allowed. Two arcs $v \to w$ with distinct labels are allowed and are counted separately. In the terms of [**?**, §3.1], a carry graph is a labelled graph with at most one edge per (source, label, target) triple. Every finite labelled relation $E \subseteq V \times A \times V$ is a carry graph, and every carry graph with finite V and A is such a relation: its set of arcs.

Example 2.2 (digit graph)**.** Let $a \geq 2$ and $M \geq 1$. The *digit graph* $D_{a,M}$ has vertex set $\mathbb{Z}/M$, alphabet $\{0, \dots, a-1\}$, and an arc $r \xrightarrow{d} ar+d$ whenever both r and $ar + d$ are units. If $x_n = \lfloor \xi a^n \rfloor$ and d_n is the n-th base-a digit

of ξ, then $x_{n+1} = ax_n + d_n$. So the residues of x_n walk along $D_{a,M}$ for as long as they are units, and they read the digits of ξ.

Example 2.3 (cell graph)**.** Let $p > q \geq 1$, $\nu \in \mathbb{R}$, $D \geq 1$, and let $S \subset \mathbb{Z}$ be finite. The cell graph has vertex set $\{0, \dots, D-1\}$, alphabet S, and an arc $c \xrightarrow{s} c'$ when the interval $\big(p\,[c/D, (c+1)/D) - (p-q)\nu - s\big)/q$ meets $[c'/D, (c'+1)/D)$. By the carry identity (1.1), the cells of the fractional parts y_n walk along it.

Constructions. For a carry graph X:

- the *reverse* $X^\top$ turns every arc around and keeps its label;
- the *restriction* $X|_S$ to $S \subseteq V$ keeps the arcs with both ends in S;
- for $g\colon A \to B$, the *relabelling* g_*X has $v \xrightarrow{b} w$ iff $v \xrightarrow{a} w$ in X for some a with $g(a) = b$; forgetting labels is the case $B = \{*\}$.

A *homomorphism* $f\colon X \to Y$ of carry graphs over the same alphabet is a map on vertices sending arcs to arcs with the same label: $v \xrightarrow{a} w$ implies $f(v) \xrightarrow{a} f(w)$.

Example 2.4. If $M' \mid M$, reduction modulo M' is a homomorphism $D_{a,M} \to D_{a,M'}$, because ring homomorphisms send units to units and preserve $ar+d$. This is the map behind the ladder of Chapter 3.

2.2 Walks, infinite paths, words

A *walk* of length n is a sequence $g_0 \to g_1 \to \cdots \to g_n$ in the underlying digraph. An *infinite path* is a pair of sequences (s, ℓ) with $s_n \xrightarrow{\ell_n} s_{n+1}$ for all n. We say that *reading* the word $w = a_1 \cdots a_k \in A^*$ from v can end at u if there is a walk $v = t_0 \xrightarrow{a_1} t_1 \to \dots \xrightarrow{a_k} t_k = u$. The word w is a *word of X at v* if reading it from v ends somewhere. These words form the *label language* $\mathcal{L}_v(X)$.

Proposition 2.5. *(a) Reading $w_1 w_2$ from v ends at u iff reading w_1 ends at some t from which reading w_2 ends at u.*

(b) $\mathcal{L}_v(X)$ contains the empty word and is closed under prefixes.

(c) For a finite $S \subseteq V$, the set $\mathrm{ends}(S, w)$ *of vertices where reading w from S can end is finite. It is computed letter by letter by* $\mathrm{ends}(S, aw) = \mathrm{ends}\big(\bigcup_{v \in S} \mathrm{succ}(v, a),\, w\big)$*, and $w \in \mathcal{L}_v(X)$ iff* $\mathrm{ends}(\{v\}, w) \neq \emptyset$.

Proof. (a) is induction on w_1. (b) follows from (a). (c) is induction on w: a finite union of finite sets is finite. □

Part (c) means that membership in $\mathcal{L}_v(X)$ is decidable whenever equality of vertices is, even for infinite V. The sets $\mathrm{ends}(\{v\}, w)$ are the states of the subset construction of Chapter 6.

2.3 Counting paths and words

Let A be finite. The number of labelled paths of length n from v is

$$N_v(0) = 1, \qquad N_v(n+1) = \sum_{a \in A} \sum_{u \in \mathrm{succ}(v,a)} N_u(n),$$

and $\mathcal{L}_v^n$ denotes the set of words of length n at v.

Definition 2.6. X is *right-resolving* if $|\mathrm{succ}(v,a)| \le 1$ for all v, a [**?**, Def. 3.3.1]. X is *functional* if at most one arc leaves each vertex.

Proposition 2.7. *(a)* $|\mathcal{L}_v^n| \le N_v(n)$.

*(b) If X is right-resolving, then $|\mathcal{L}_v^n| = N_v(n)$: a path from v is determined by its label [**?**, Prop. 3.3.9].*

(c) If X is functional, then $N_v(n) \le 1$.

Proof. Both sides of (a) satisfy the same recursion, with $\le$ on the left because the words $a\,w'$ obtained from different successors can coincide. For (b), words with different first letters are different. For a fixed first letter there is at most one successor, and prefixing a is injective, so the recursion holds with equality. For (c), induct on n: at most one term of the outer sum is non-empty, and it has one summand. □

2.4 Kőnig's lemma

Theorem 2.8 (Kőnig)**.** *Let A be finite and $v \in V$. If there are walks of every length starting at v, then there is an infinite path starting at v. In particular, if $\mathcal{L}_v(X)$ contains words of every length, then X has an infinite path (s, ℓ) with $s_0 = v$.*

Proof. Call a vertex *good* if walks of every length start at it. Let v be good, and suppose no successor of v is good. For each $w \in \mathrm{out}(v)$ choose N_w such that no walk of length N_w starts at w, and put $N = \max_w N_w$. This maximum exists because $\mathrm{out}(v)$ is finite. A walk of length $N+1$ from v passes through some $w \in \mathrm{out}(v)$ and continues with a walk of length $N \ge N_w$ from w. Its prefix of length N_w is then a walk of length N_w from w, a contradiction. So every good vertex has a good successor, and choosing one repeatedly gives the infinite path. Labels are chosen arc by arc. □

Mathlib proves Kőnig's lemma for strongly atomic orders and inverse systems (`Order/KonigLemma`). For carry graphs the direct argument above is shorter than either reduction. A sharper version, with an injective path, is Theorem 3.10 in the next chapter.

Notes

The definitions follow [**?**, §2.2, §3.1] and [**?**, §1.2]. The only difference is that vertex sets may be infinite, with finitely many arcs per label. Right-resolving presentations are the subject of [**?**, §3.3]. The digit graph is the coprimality subshift of [**?**, **?**], and the cell graph goes back to [**?**].

Lean dictionary

Text	Lean (`CarryGraph/Basic.lean`, `Path.lean`)
Definition 2.1	`CarryGraph`, `Adj`, `under`, `out`
finite relations, arc sets	`ofRel`, `ofFinset`, `arcs`, `ofFinset_arcs`
constructions	`reverse`, `restrict`, `relabel`, `forget`, `Hom`
Example 2.2, 2.4	`CarryBridge.digitGraph`, `CarryBridge.projHom`
walks, infinite paths	`IsWalk`, `IsIPath`
Proposition 2.5	`Reads`, `IsWord`, `reads_append`, `IsWord.of_append`, `ends`, `isWord_iff_ends_nonempty`
Proposition 2.7	`npaths`, `words`, `card_words_le_npaths`, `card_words_eq_npaths`, `Functional.npaths_le_one`
Theorem 2.8	`exists_walk_of_forall_hasWalk`, `exists_iPath_of_forall_isWord`

Chapter 3

Reachability, strong components and ranks

Most of the finite witnesses in the problems of Chapter 1 are *ranks*: functions $\rho\colon V \to \mathbb{N}$ that never increase along an arc. This chapter develops ranks from the strong-component structure of [**?**, §1.5]. The main results characterise when a rank exists that leaves at most one rank-keeping successor (Theorem 3.4), show that it forces eventual periodicity (Theorem 3.5), and give the peel-rank criterion that locates all cycles (Theorem 3.6).

3.1 Reachability

Write $u \rightsquigarrow v$ if there is a walk from u to v, and $u \rightsquigarrow_n v$ if there is one of length exactly n. Two vertices are in the same *strong component* if $u \rightsquigarrow v$ and $v \rightsquigarrow u$. This is an equivalence relation. A vertex is *cyclic* if a walk of positive length leads from it back to itself, that is, if it has a successor t with $t \rightsquigarrow v$. Cyclic vertices form whole strong components. Homomorphisms preserve $\rightsquigarrow$ and send cyclic vertices to cyclic vertices.

Computing reachability. Let A be finite. The *ball* of radius k around v is $B_0 = \{v\}$, $B_{k+1} = B_k \cup \mathrm{out}(B_k)$. So $w \in B_k$ iff $v \rightsquigarrow_n w$ for some $n \le k$.

Proposition 3.1. *If $B_{k+1} = B_k$, then B_k contains every vertex reachable from v. If V is finite, this happens for some $k \le |V|$. Hence the vertices reachable from v are exactly those in $B_{|V|}$, and reachability is decidable.*

Proof. A ball that equals its successor ball is closed under out, so it contains every vertex reachable from v. If $B_{k+1} \neq B_k$ for all $k \le |V|$, then $|B_k| \ge k+1$ for $k \le |V|$ by induction, and $|B_{|V|}| > |V|$ is impossible. Balls increase with k. □

Mathlib has no decision procedure for the reflexive–transitive closure of a relation, so Proposition 3.1 is what makes $\rightsquigarrow$ usable by `decide`.

3.2 The two canonical ranks

Let V and A be finite. The *descendant count* and the *predecessor count* are

$$\mathrm{desc}(v) = |\{u : v \rightsquigarrow u\}|, \qquad \mathrm{preds}(v) = |\{u : u \rightsquigarrow v\}|.$$

Proposition 3.2 (acyclic ordering, [**?**, Prop. 1.4.3, §1.5])**.** *Let $s \to t$ be an arc.*

(a) $\mathrm{desc}(t) \leq \mathrm{desc}(s)$*, with equality iff $t \rightsquigarrow s$, i.e. iff s and t lie in one strong component.*

(b) $\mathrm{preds}(s) \leq \mathrm{preds}(t)$*. If s and t lie in different components, then $\{u : u \rightsquigarrow s\} \subsetneq \{u : u \rightsquigarrow t\}$.*

(c) desc *of the reverse graph is* preds*.*

Proof. (a) Descendants of t are descendants of s. If the two finite sets have equal size, they are equal, so s, a descendant of itself, is a descendant of t. Conversely, if $t \rightsquigarrow s$ the inclusion goes both ways. (b) is dual, with t a predecessor of itself but not of s. (c) holds because a walk in the reverse graph is a reversed walk. □

So desc is non-increasing along arcs and strictly decreasing exactly across components. In other words, it is a rank form of the acyclic ordering of the strong component digraph. Equivalently, the condensation is acyclic. The predecessor count is the dual ordering, increasing along arcs. One problem family uses one convention and another uses the other. This book uses non-increasing ranks throughout.

3.3 Ranks and branching

Definition 3.3. A *rank* of X is a $\rho\colon V \to \mathbb{N}$ such that

(R1) $\rho(t) \leq \rho(s)$ for every arc $s \to t$;

(R2) every vertex s has at most one successor t with $\rho(t) = \rho(s)$.

A *labelled rank* replaces (R2) by: at most one arc $s \xrightarrow{a} t$ with $\rho(t) = \rho(s)$. X has *no branching* if no vertex has two distinct successors in its own strong component (labelled: two distinct arcs).

Theorem 3.4. *For finite V and A, X has a rank iff it has no branching, and then* desc *is a rank. The labelled version holds likewise. For infinite V, if every vertex has finitely many descendants, the same holds with* $\mathrm{desc}(v) = |\{u : v \rightsquigarrow u\}|$.

Proof. A rank is constant on strong components: along a walk it cannot increase, and a walk leads back. So two successors of s in the component of s are both rank-keeping, and (R2) makes them equal. Conversely, without branching, desc satisfies (R1) by Proposition 3.2(a). By the same proposition, a successor that keeps desc lies in the component of s, and there is at most one of those. For infinite V, use $|\cdot|$ on finite sets in the same argument. □

No branching means that every strong component containing an arc is a single cycle [**?**, Def. 2.2.13]. Chapter 5 shows that this is exactly zero entropy. Here we prove its first consequence.

Theorem 3.5 (eventual periodicity)**.** *Let ρ be a rank and $s_0 \to s_1 \to \cdots$ an infinite walk such that only finitely many vertices are reachable from s_0. Then there are n_0 and $L > 0$ with $s_{m+L} = s_m$ for all $m \geq n_0$. If ρ is a labelled rank and (s, ℓ) an infinite path, the labels ℓ are eventually periodic with the same period.*

Proof. The sequence $\rho(s_n)$ does not increase, so it is constant from some n_1 on. From then on every step is rank-keeping, so by (R2) s_{m+1} is determined by s_m. The walk stays in a finite set, so $s_p = s_q$ for some $n_1 \leq p < q$. By induction $s_{m+(q-p)} = s_m$ for all $m \geq p$. In the labelled case the arcs $s_m \xrightarrow{\ell_m} s_{m+1}$ and $s_m = s_{m+L} \xrightarrow{\ell_{m+L}} s_{m+L+1} = s_{m+1}$ are both rank-keeping, so $\ell_{m+L} = \ell_m$. □

Theorem 3.5 is the engine behind the digit problem of Chapter 1. If the residues of $\lfloor \xi a^n \rfloor$ modulo M are eventually units, they form an infinite walk in $D_{a,M}$. A rank of $D_{a,M}$ makes the walk eventually periodic. For $a \leq M$ the digit is determined by consecutive residues, so the digit word is eventually periodic too, and [**?**, Theorem 4] turns that into infinitely many composite terms. The theorem is local: it uses only the part of the graph reachable from the walk.

3.4 Peel ranks

Theorem 3.6 (peel rank)**.** *Let $C \supseteq \{\text{cyclic vertices}\}$ and $K \subseteq V$. Suppose $\rho\colon V \to \mathbb{N}$ satisfies $\rho(t) \leq \rho(s)$ for every arc $s \to t$ inside C, and $\rho(t) < \rho(s)$ for every such arc with $s \notin K$. Then every cyclic vertex lies in K. The same holds with all inequalities and arcs reversed: $\rho(s) \leq \rho(t)$ inside C, strictly if $t \notin K$.*

Proof. Let $r \notin K$ be cyclic, with a closed walk $r \to t \rightsquigarrow r$. Every vertex of this closed walk is cyclic, hence in C. The first step drops the rank strictly and no later step raises it, so $\rho(r) \leq \rho(t) < \rho(r)$. For the reversed statement, apply this to the reverse graph, which has the same cyclic vertices. □

Corollary 3.7 (rungs)**.** *Let $f\colon X \to Y$ be a homomorphism and K' a set containing every cyclic vertex of Y. If ρ is a peel rank for K on $C = f^{-1}(K')$, then every cyclic vertex of X lies in K.*

Proof. Homomorphisms send cyclic vertices to cyclic vertices, so $f^{-1}(K')$ contains every cyclic vertex of X. Then apply Theorem 3.6. □

With f the reduction $D_{a,M} \to D_{a,M'}$ (Example 2.4), Corollary 3.7 is one rung of a ladder of moduli $M_1 \mid M_2 \mid \cdots$. Each rung only needs to check the rank on the fibre over the previous core. This is how the base-7 case of the composite problem was checked.

Proposition 3.8 (core form)**.** *If every cyclic vertex lies in K and no vertex of K has two distinct successors in K, then X has no branching. For finite V, A it therefore has a rank.*

Proof. Two successors of v in the component of v are cyclic, as is v, so all three lie in K. □

The converse fails. A rank can exist while the set of units is not deterministic: in base 4 modulo 6 a rank exists, but the units do not satisfy the hypothesis of Proposition 3.8 with K the set of all units.

Proposition 3.9 (rank depth)**.** *Along a walk $g_0 \to \cdots \to g_n$ on which ρ never increases, the number of steps with $\rho(g_{i+1}) < \rho(g_i)$, plus $\rho(g_n)$, is at most $\rho(g_0)$.*

Proof. Induction on n: each strict drop lowers ρ by at least one. □

3.5 Geodesic Kőnig

Theorem 3.10. *Let A be finite. If infinitely many vertices are reachable from v, then there is an infinite walk $v = s_0 \to s_1 \to \cdots$ whose vertices are pairwise distinct. In fact* $\operatorname{dist}(v, s_n) = n$ *for all n.*

Proof. Let $\operatorname{dist}(v, w)$ be the length of a shortest walk from v to w. Call an arc $u \to w$ *geodesic* if u is reachable and $\operatorname{dist}(v, w) = \operatorname{dist}(v, u) + 1$. Every reachable w is reached from v by geodesic arcs in exactly $\operatorname{dist}(v, w)$ steps. To see this, take the last step $u \to w$ of a shortest walk. Then $\operatorname{dist}(v, u)$ is at most the length of the walk minus one, and at least $\operatorname{dist}(v, w) - 1$, so the arc is geodesic, and we induct. Balls are finite, so for each n some reachable w has $\operatorname{dist}(v, w) \geq n$. Hence the geodesic subgraph has walks of every length from v. Theorem 2.8 gives an infinite geodesic walk, along which the distance increases by one at each step. □

This is the abstract half of the finiteness theorem for follower automata in Chapter 6: an infinite automaton has an infinite path that never revisits a state.

Notes

Strong components and their acyclic ordering are [**?**, §1.5]. The two rank conventions appear in the problems as descendant counts (the digit and divisor problems) and as predecessor counts (the word-graph certificates of [**?**]). Theorem 3.5 for finite graphs and Theorem 3.6 come from the digit problem. Theorem 3.4 is their converse, and it shows that a rank witness exists exactly when the graph has zero entropy (Chapter 5).

Lean dictionary

Text	Lean (`CarryGraph/Reach.lean`, `Rank.lean`)
reachability, components, cycles	`Reach`, `ReachIn`, `SameSCC`, `sameSCC_equivalence`, `Cyc`, `cyc_iff`
homomorphisms	`Hom.reach_map`, `Hom.cyc_map`
Proposition 3.1	`ball`, `mem_ball`, `exists_ball_stable`, `reachSet`, `mem_reachSet`, `decReach`
Proposition 3.2	`desc`, `preds`, `desc_eq_iff`, `desc_lt_of_not_reach`, `preds_ssubset`, `desc_reverse`
Definition 3.3	`IsRank`, `IsLRank`, `NoBranch`, `LNoBranch`
Theorem 3.4	`IsRank.noBranch`, `noBranch_isRank_desc`, `exists_isRank_iff_noBranch`, `noBranch_isRank_ncard`
Theorem 3.5	`IsRank.evPer`, `IsLRank.evPer`
Theorem 3.6	`cyc_mem_of_peelRank`, `cyc_mem_of_peelRank_rev`
Corollary 3.7	`CarryBridge.coreClosed_of_ladder'`
Proposition 3.8	`noBranch_of_cyc_det`
Proposition 3.9	`drops_add_le`
Theorem 3.10	`exists_injective_walk_of_infinite_reach`

Chapter 4

Peeling and the core

To be written.

Chapter 5

Growth and entropy

To be written.

Chapter 6

Labels, languages and sofic shifts

To be written.

Chapter 7

Splittings, homomorphisms and products

To be written.

Chapter 8

Covers of dynamical systems

To be written.

Chapter 9

Affine carry graphs

To be written.

Appendix A

Lean dictionary

To be assembled from the per-chapter tables.

Appendix B

What Mathlib and ForMathlib supply

To be written (M7).